PhisTools

A Policy-Aware Phishing Risk Analysis and Prevention-Support Tool

Technical Project Report

Network Information Security (NIS)

Prepared by
Devansh Agarwal (23BIT0028)
Nakul Thombare (23BIT0013)
Samarth Khandelwal (23BIT0051)

Faculty / Guide
Dr Shantharajah S P

Academic Year
2025–2026

Abstract

PhisTools is a Streamlit-based phishing assessment prototype developed for the Network Information Security course. The system accepts email subject and body text, optional header data, one or more URLs, and an optional remote attachment URL. It combines machine-learning predictions with security heuristics, policy-aware thresholds, and analyst-style interpretation to produce a final risk score, threat level, verdict, attack-type classification, and response guidance. The project is intentionally designed as a phishing triage workflow, not only as a standalone classifier. Its main academic value is the integration of ML evidence, security logic, logging, explainability, and controlled runtime robustness into one compact prototype.

Executive Summary

Phishing is a multi-signal security problem. A suspicious email may contain urgent language, spoofed sender information, deceptive links, or attachment lures at the same time. Because of this, a single classifier is often not enough for useful security interpretation. PhisTools addresses this by combining email-content models, URL analysis, header anomaly checks, NIS heuristics, policy-aware risk scoring, attack categorization, and recommended defensive actions. The final output is therefore closer to a lightweight analyst-support tool than a simple prediction page.

1. Introduction and Problem Statement

Phishing remains one of the most common attack vectors in academic institutions, enterprises, and public-facing digital services.
Modern phishing campaigns do not rely on only one trick. In many cases, the attacker mixes social-engineering language, lookalike domains, sender manipulation, shortened links, fake login portals, and attachment-based lures.
A basic detector may produce a label such as phishing or safe, but such a label alone is often not enough for practical security decision-making.

PhisTools was developed to address this gap at an academic prototype level. The project combines machine-learning outputs with Network Information Security logic so that the result becomes more useful for triage, explanation, and response.
Instead of ending at a raw classification, the system tries to answer four questions: whether the input is suspicious, why it is suspicious, what kind of attack it resembles, and what the user should do next.

2. Need for the Project and Objectives

There is a clear need for a phishing-analysis tool that does more than text classification. In practical security, suspicious emails are judged using multiple signals such as wording, sender authenticity, link structure, urgency, and likely attacker objective.
This makes phishing a good NIS problem because it requires risk interpretation under uncertainty and not only ML prediction.

The primary objective of PhisTools is to build a practical phishing risk-analysis prototype that fuses machine-learning evidence with security logic.
The supporting objectives are to provide explainable output, include header and spoofing checks, inspect suspicious URLs, classify likely attack types, support policy-based verdicting, and store scan history and user feedback.

The present scope is prototype-oriented. The system is not designed as a production email gateway, full SPF/DKIM/DMARC validation platform, or malware detonation sandbox.

3. System Overview

From the user perspective, PhisTools behaves as a single-interface phishing triage application. A user can provide suspicious email content, optional raw headers, one or more URLs, and optionally a remote attachment URL. The application then processes these inputs through the runtime pipeline and presents a structured output instead of a single raw probability.

Main inputs accepted by the system are:

- email subject text
- email body text
- optional raw headers
- optional multiple URLs, entered one per line
- optional remote attachment URL

Main outputs generated by the system are:

- email model probabilities
- URL probabilities, both per-link and averaged
- header risk assessment and detected issues
- final risk score on a 0-100 scale
- threat level and verdict
- attack-type categories
- recommended preventive or response actions
- grouped explanatory indicators
- scan log and optional feedback record

In simple terms, the workflow is: user input -> model inference -> NIS feature extraction -> risk fusion -> verdicting -> explanation -> logging.

4. NIS Positioning of the Project

A key requirement of this report is to position the project correctly. PhisTools should be framed primarily as an NIS project and not merely as a machine-learning exercise. This is justified by the system design. ML contributes predictive evidence, but the final value of the project comes from security interpretation and decision support.

NIS-oriented components implemented in the project include:

- header and spoofing analysis
- suspicious URL and domain inspection

- policy-aware risk scoring
- threat-level mapping
- attack-type categorization
- recommended prevention and response actions
- logging and feedback capture
- optional threat-intel checks
- optional remote attachment analysis

In other words, ML helps the project estimate whether the content looks phishing-like, while the NIS logic interprets how risky the event is from a security perspective and what action should follow. Because of this, the project behaves like a lightweight phishing triage tool.

5. ML Positioning and Hybrid Design

The ML layer of PhisTools includes an email logistic-regression model, an email LSTM model, an optional hybrid/meta model for email fusion, and an active URL model. These models provide probabilities, but they do not directly define the complete system output. The final verdict is obtained after these scores are combined with security heuristics and policy rules.

The role of each model is summarized below.

Model / Artifact	Purpose	Runtime Role	Status
Email logistic model	Scores phishing-likeness from TF-IDF text features	Email-side evidence	Active
Email LSTM model	Learns sequence-level phishing language patterns	Email-side evidence	Active
Hybrid/meta model	Combines logistic and LSTM probabilities	Final email probability when available	Optional but used
URL model	Scores suspiciousness of canonicalized URL and host input	URL-side evidence	Active

6. Runtime Architecture and Working Flow

The active entrypoint of the project is webapp.py. At startup, the application prepares runtime compatibility settings for model loading, imports helper modules from the utils directory, and then executes the cached load_models() path. The Streamlit interface then displays model status, policy controls, input fields, and analysis actions.

The active runtime flow is as follows:

- startup and model loading from the models directory
- user input collection through the Streamlit interface

- email probability computation through predict_email_models()
- header risk extraction through analyze_headers()
- URL scoring through predict_url_ml()
- NIS feature extraction from email and URL context
- offline intelligence enrichment and optional online checks
- risk-score computation and runtime calibration
- policy-aware threat mapping and final verdicting
- attack-type classification and action recommendation
- log writing and optional feedback persistence

This design is important because it clearly separates retraining-only files from runtime-critical files. During normal application use, the train scripts are not executed. The runtime depends only on webapp.py, the supporting utility modules, and the saved model artifacts.

7. Important Files and Their Roles

The repository can be understood in three groups: runtime-critical files, retraining files, and experimental or legacy files.

7.1 Runtime-Critical Files

- webapp.py - Active entrypoint and main orchestrator of the UI, model loading, inference, scoring, verdicting, and logging.
- utils/nis_features.py - Extracts email and URL security indicators used in scoring and explanation.
- utils/risk_scoring.py - Computes the base risk score and compact reasons.
- utils/scan_logger.py - Persists scan history to logs/scan_logs.csv.
- utils/threat_intel.py - Provides offline intelligence and optional online checks.
- utils/attachment_file_analyzer.py - Analyzes downloaded remote files using MIME and file-based heuristics.

7.2 Retraining Files

- train_logistic_model.py - Generates the email logistic model and preferred vectorizer.
- train_deep_model.py - Generates the email LSTM model, tokenizer files, and metadata.
- train_hybrid_model.py - Generates the hybrid meta model for email fusion.
- train_all.py - Integrated email training path.
- train_url_ml_model.py - Generates the active runtime URL model artifact.
- merge_datasets.py - Prepares the merged URL dataset used in active URL retraining.

7.3 Experimental or Legacy Files

Some additional scripts remain in the repository for earlier experiments, visual training runs, toy models, or safe archival history. These files help explain the project's evolution, but they are not required for normal application execution.

8. Implemented Features

Feature	Purpose	Type	Runtime Status
Email ML scoring	Computes logistic, LSTM, and fused probabilities from email text	ML / Hybrid	Active
URL scoring	Generates per-URL and averaged URL risk values	ML + NIS context	Active
Header analysis	Checks spoofing-style anomalies in provided headers	NIS	Active
Risk scoring	Combines evidence into a final 0-100 score	Hybrid	Active
Policy mode	Adjusts suspicious and phishing thresholds	NIS	Active
Attack typing	Maps evidence to categories such as credential harvesting or spoofing	NIS	Active
Recommendations	Provides prevention and response guidance	NIS	Active
Logging and feedback	Stores scan history and feedback records	Operational	Active
Remote attachment scan	Fetches and analyzes remote files when used	NIS operational	Optional
Threat-intel checks	Adds offline and optional online intelligence context	NIS enrichment	Optional

9. Heuristics, Security Logic, and Risk Fusion

A central idea of PhisTools is the use of NIS heuristics. In this project, heuristics means rule-based security indicators designed from phishing knowledge, not learned automatically by the models. Examples include urgency cues, credential-harvesting cues, header mismatch signals, suspicious URL-domain patterns, shortener flags, suspicious TLD indicators, IP-host indicators, and related infrastructure clues.

The final risk score is therefore not a raw model output. It is a hybrid score built from machine-learning probabilities plus security logic. The raw score is computed in the risk-scoring module and then adjusted by a calibration layer in the runtime so that weak evidence does not over-inflate the result and strong combined signals escalate more appropriately.

Main inputs to the final score are:

- email-side probability from logistic, LSTM, or hybrid email path
- URL-side probability from the active URL model
- header or spoofing evidence
- email and URL heuristics extracted from security logic
- offline intelligence and policy-threshold interpretation

10. Datasets, Training Pipeline, and Artifacts

The email training path uses CEAS_08.csv, while the URL training path uses a merged URL dataset built from phishing URL sources. The merged dataset is prepared through merge_datasets.py and then consumed by train_url_ml_model.py.

A recent retraining step improved the URL model by adding structural URL features in addition to lexical URL representation. These features include host length, full URL length, subdomain depth, digit ratio, hyphen count, suspicious keyword count, IP-host flag, suspicious TLD flag, punycode flag, path depth, and shortener flag. Probability calibration was also added while keeping runtime compatibility with the same artifact name and predict_proba interface.

It is important to note that the train scripts are used only for artifact generation. During ordinary application use, the runtime loads the already saved model files from the models directory.

11. Results and Observed Performance

Manual testing and iterative refinement show that the system performs best when multiple suspicious signals align. In such cases, the project gives useful results through a combination of model evidence, attack classification, and response guidance.

Observed strengths include:

- strong behavior in clear phishing cases with urgency, credential, and spoofing cues
- useful attack-type categorization
- practical recommended actions
- clear grouped explainability for demo and review purposes
- policy-aware interpretation rather than a raw yes/no output

Observed limitations include:

- URL calibration still imperfect for some trusted enterprise or support domains
- header interpretation depends on the realism and completeness of provided headers

- remote attachment scanning depends on remote host behavior and network conditions
- full forensic-grade header parsing is outside current scope

12. Improvements Made During Iteration

The project was improved after runtime testing. The goal of the improvement phase was not to redesign the system but to make the existing workflow more stable, explainable, and defensible during review.

- Header analysis was improved with confidence-aware handling for manual or incomplete headers.
- A calibration layer was added so weak evidence contributes less aggressively while combined strong evidence escalates more appropriately.
- Explainability was grouped under Email, Header/Spoofing, URL/Domain, Threat Intel, and Final Risk Reasoning.
- Remote attachment handling was made more robust through safer validation, clearer failure messages, redirect checks, timeout handling, and cleanup improvement.
- The URL model training pipeline was upgraded with structural features, calibration, and benign-domain anchoring to reduce holdout false positives while keeping runtime compatibility.

13. Novelty and Distinctive Value

The project is not novel because phishing detection itself is new. The value of PhisTools is in the integration. It combines ML and NIS reasoning into a policy-aware phishing risk-analysis workflow. It does not stop at classification; it adds grouped explanation, attack-type mapping, operational logging, and action guidance. Because of this, the system is more meaningful as an NIS educational prototype than a single-model demo.

14. Limitations and Future Work

PhisTools should be presented honestly as a strong academic prototype, not a production-hardened enterprise system. Its remaining limitations are important and should be stated directly.

- trusted-domain false positives are reduced but not fully solved
- header analysis still depends on the quality of user-provided header text
- remote attachment scanning remains dependent on remote server policy and network conditions
- online threat-intel depends on API keys and environment access
- the project does not implement full RFC-grade email forensics or malware sandboxing

Useful future extensions are:

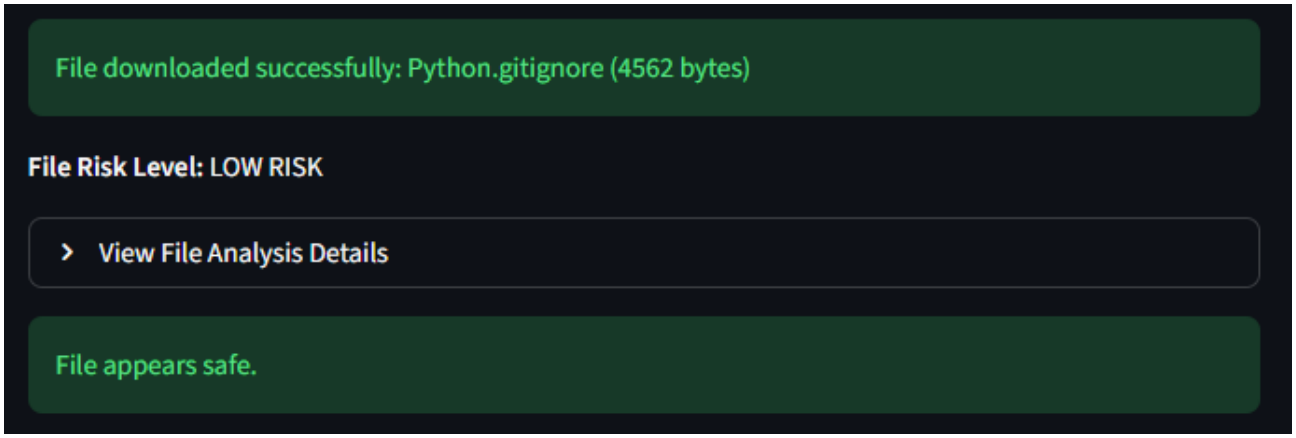
- larger curated benign URL corpus, especially for academic, enterprise, and government domains
- domain-level threshold tuning and improved reputation fusion
- optional SPF, DKIM, and DMARC enrichment
- more advanced header parsing
- stronger packaging and dependency pinning for reproducible deployment

15. Screenshots









Conclusion

PhisTools demonstrates how an educational phishing prototype can be improved from a simple classifier into a more meaningful Network Information Security workflow. Its main contribution is the combination of machine-learning evidence with security heuristics, policy-aware scoring, attack categorization, grouped explainability, operational logging, and response guidance. Although model calibration and deployment hardening remain future tasks, the project is already valuable as an NIS submission and as a practical learning artifact for the team.